MACHINE LEARNING PROJECT

HomeValue

A Complete Guide to the House Price Prediction System

From first principles to a running web application.
Which algorithm was used, how it works, and exactly why it was chosen.

Selected model: Random Forest Regressor
Held-out test R-squared 0.8676 - typical error $19,039
Dataset: Ames Iowa Residential Housing Dataset (2006 to 2010)

How to read this guide

This document explains the project from the ground up. It assumes you are comfortable running a Python script, but it assumes **no prior machine-learning knowledge**. Every term is defined the first time it appears, and every number quoted here was produced by the code in this repository - none of it is illustrative.

If you only read one chapter, read **Chapter 11**. It answers the question this guide is built around: which algorithm does the system use, and why that one and not the simpler alternative.

If you are...	Start with
Completely new to machine learning	Chapters 1 to 3, then 9 to 11
Looking for the model choice and justification	Chapters 9 to 12
Interested in the numbers and what they mean	Chapters 14 to 16
Here for the application and how to run it	Chapters 17 to 22
Checking what the system cannot do	Chapter 23

> **Everything in here is reproducible**
>
> Delete the model files, run `python -m src.train_model`, and every figure in this guide regenerates. The dataset is fetched from a public source by the code itself, so there is nothing to download by hand.

Contents

How to read this guide . . . 2

What machine learning actually is . . . 5

The problem we are solving . . . 5

Where the data came from . . . 6

Looking at the data first . . . 8

Cleaning: what we removed and exactly why . . . 11

Feature engineering: building the ten-column schema . . . 11

The train/test split, and why leakage ruins everything . . . 12

Algorithm one: Linear Regression . . . 13

Algorithm two: Decision Trees . . . 14

Algorithm three: Random Forest . . . 14

Which algorithm was chosen, and why . . . 15

The preprocessing pipeline in detail . . . 17

How accuracy is measured here . . . 18

Results . . . 18

What the model learned . . . 20

How a prediction request flows . . . 22

The web application . . . 22

The prediction dashboard . . . 23

Validation and error handling . . . 24

Testing 24

Running the project yourself 25

Project structure 26

What this system cannot do 26

What would be built next 27

Glossary 27

Appendix: JSON API reference 28

Appendix: the ten-column data schema 28

Part I - Foundations

1. What machine learning actually is

Traditional programming works like this: you write rules, feed in data, and get answers out. Machine learning inverts it. You feed in data *and* the answers, and the computer works out the rules.

```
Traditional:   DATA + RULES        -> ANSWERS
Machine learning: DATA + ANSWERS   -> RULES

In our case: 1,458 sold homes + their sale prices -> a rule that
             estimates the price of a home it has never seen.
```

That learned rule is called a **model**. The process of finding it is **training**. The data with known answers is the **training set**.

The three flavours

Type	What the data looks like	Example
Supervised learning	Inputs with the correct answer attached	House features + the price it sold for
Unsupervised learning	Inputs with no answers	Grouping similar houses together
Reinforcement learning	An agent rewarded for good actions	Game-playing systems

This project is **supervised learning**. Every property in the dataset comes with the answer - the price it actually sold for.

Regression versus classification

Supervised problems split in two. **Classification** predicts a category (spam or not spam). **Regression** predicts a number. House price is a number that can take any value, so this is a **regression** problem.

> **Why this matters**
>
> The choice of regression rules out whole families of algorithms - accuracy, F1 score and confusion matrices are classification tools and are simply not applicable here. Everything that follows is measured with regression metrics.

2. The problem we are solving

Given nine things a person genuinely knows about a property - where it is, how big it is, how many rooms it has, how old it is - estimate what it would sell for.

Nine inputs were chosen deliberately. They are facts an owner or buyer knows without a surveyor: no condition scores, no renovation history, no school catchment. That constraint keeps the system usable, and Chapter 23 is honest about what it costs in accuracy.

#	Input	Type	What it means
1	location	Category	Which of 25 neighbourhoods it sits in
2	area_sqft	Number	Above-grade living area in square feet
3	lot_area_sqft	Number	Total lot size in square feet
4	bedrooms	Number	Bedrooms above grade
5	bathrooms	Number	Total bathrooms (a half bath counts 0.5)
6	property_age	Number	Age in years at the point of sale
7	floors	Number	Storeys: 1, 1.5, 2 or 2.5
8	parking	Category	No garage, 1-car, 2-car, or 3+ car
9	property_type	Category	Detached, duplex, townhouse (end or interior), two-family

The value we are predicting - the **target** - is **price**, the sale price in US dollars.

3. Where the data came from

The dataset was not downloaded by hand and is not synthetic. It is fetched by the code itself from OpenML, a public repository of machine-learning datasets.

Property	Value
Dataset	Ames Iowa Residential Housing Dataset
OpenML identifier	42165 (openml.org/d/42165)
Compiled by	Dean De Cock (Truman State University)
Published	Journal of Statistics Education, Vol. 19, No. 3 (2011)
How we retrieve it	sklearn.datasets.fetch_openml(name='house_prices')
Licence	Public Domain / CC0 Academic Teaching & Research Dataset
Market region	Ames, Iowa, United States
Observation period	2006 to 2010
Currency	USD ($)

De Cock compiled this dataset specifically to give students a modern, realistic alternative to the old Boston housing data. It records individual property sales, not regional averages, which is exactly what a per-property estimator needs.

Why not a different dataset?

Candidate	Verdict	Reason
California housing	Rejected	Census-block medians. No per-property bedroom counts or construction dates.
Boston housing	Rejected	Known ethical problems with its constructed variable. Widely deprecated.

Candidate	Verdict	Reason
KC housing / others	Rejected	Either unavailable (HTTP 404) or missing fields the application needs.
Ames, Iowa	Chosen	Real peer-reviewed per-property sales, every required field, unrestricted academic licence.

From 81 columns down to 19, then 10

The source table has 81 columns. The project keeps 19 of them - only what the application actually consumes - and then transforms those into the 10-column schema the model trains on.

Part II - Understanding and preparing the data

4. Looking at the data first

Before modelling anything, you look. The distribution of the target shapes every decision that follows - which metrics are fair, whether outliers matter, and which algorithm is likely to cope.

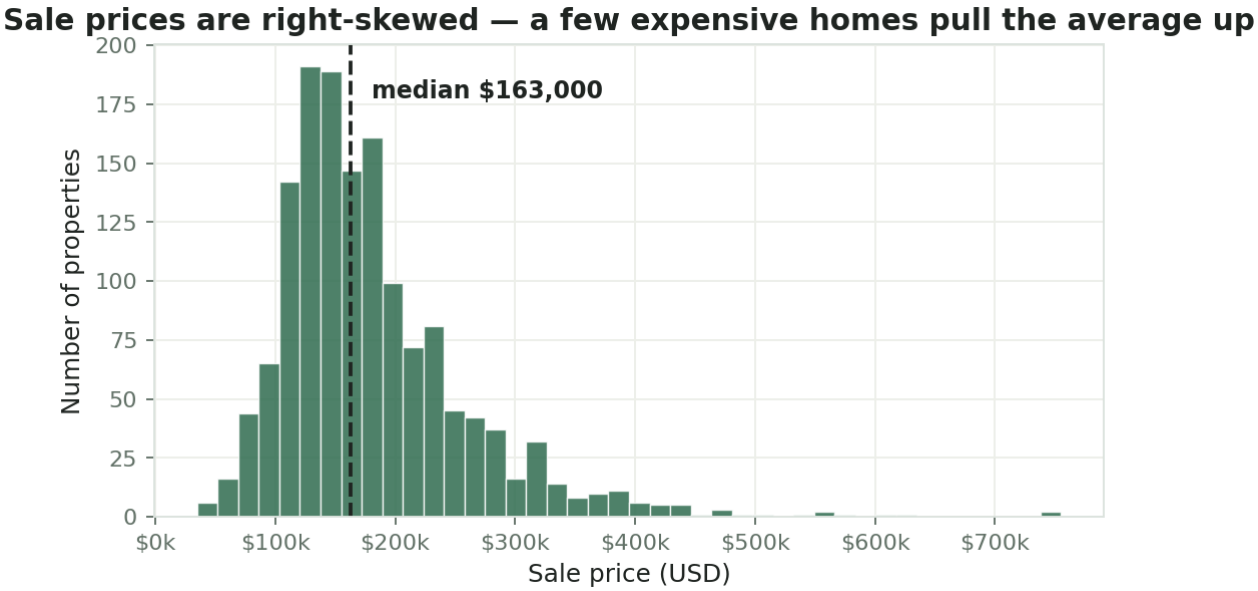


Figure 1. Sale prices across all 1,458 cleaned records. The distribution is right-skewed: most homes cluster below $250,000, with a long tail of expensive properties pulling the mean ($180,933) above the median ($163,000).

Statistic	Value	Statistic	Value
Minimum	$34,900	75th percentile	$214,000
25th percentile	$129,925	Maximum	$755,000
Median	$163,000	Mean	$180,933
Std. deviation	$79,495	Properties	1,458

Why the skew matters

Because the mean sits above the median, a metric based on average error will be pulled around by the expensive tail. That is one reason this project reports four metrics rather than one, and why RMSE is always shown alongside MAE.

Living area is the strongest single clue

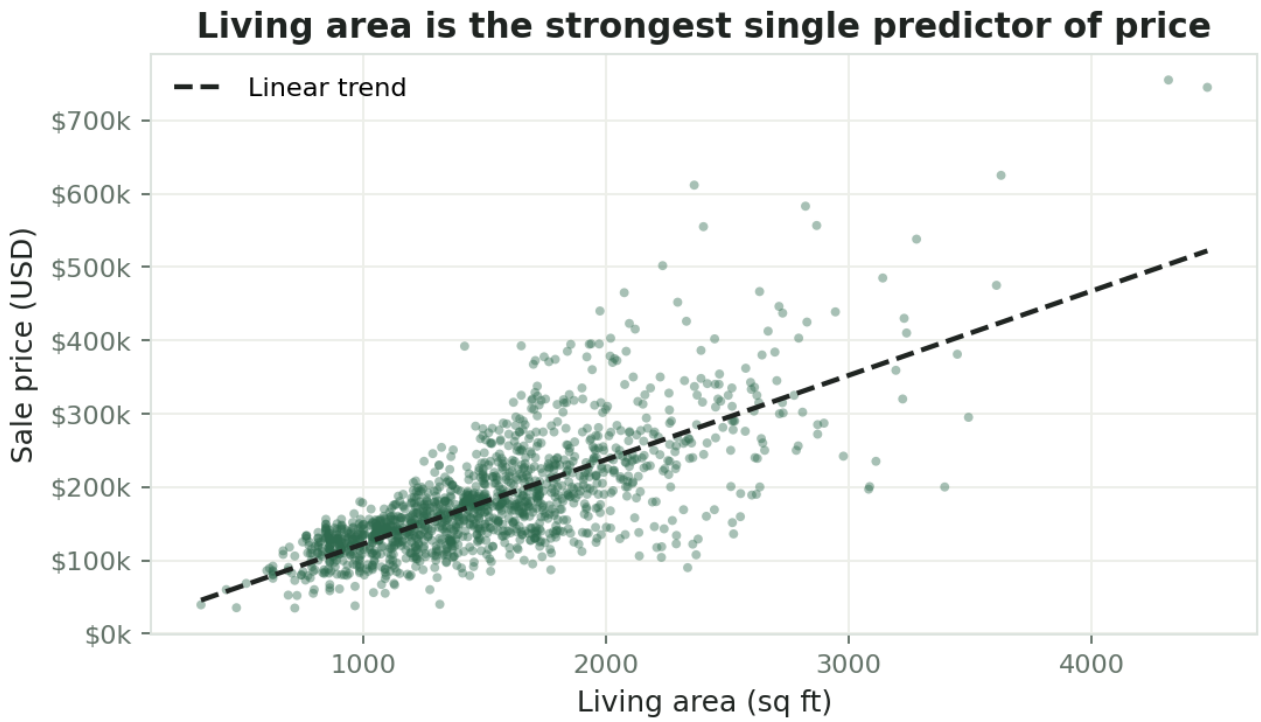


Figure 2. Living area against sale price. The relationship is clearly positive and roughly straight, with the spread widening for larger homes - a pattern a linear model can capture in direction but not in shape.

Age works the other way

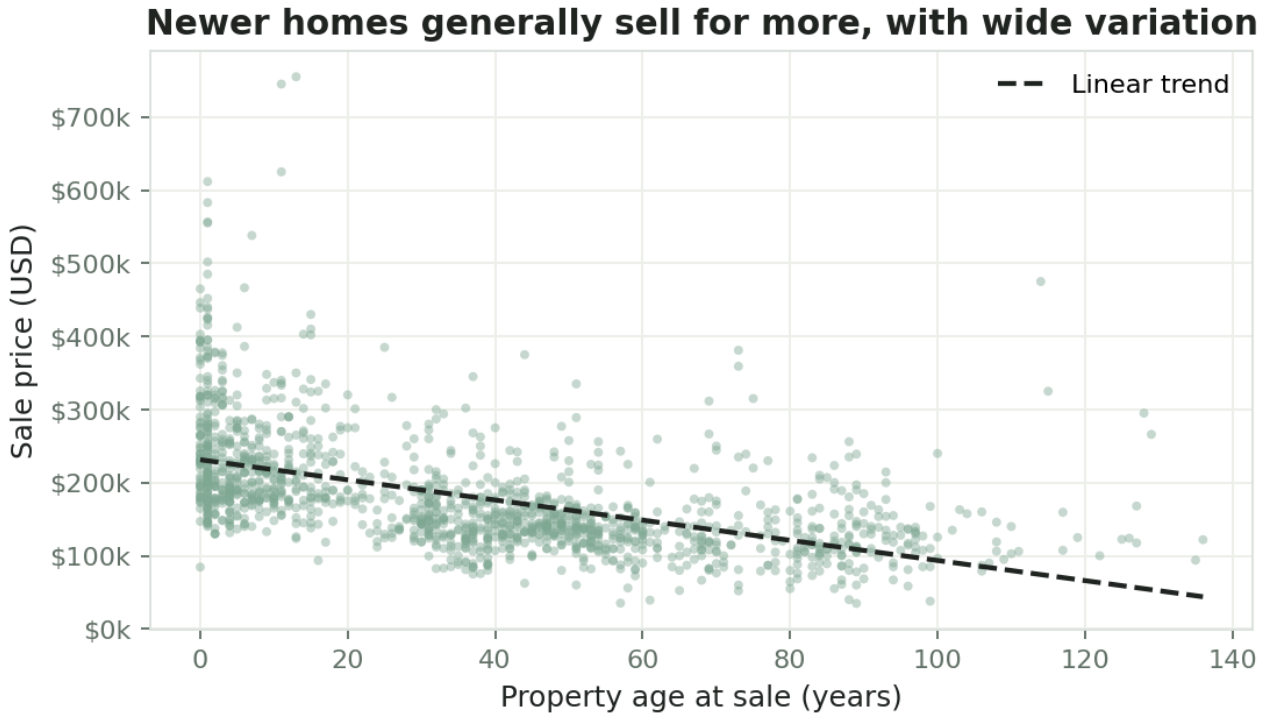


Figure 3. Property age at the point of sale against price. Newer homes fetch more on average, but the scatter is wide: an old, well-built home in a good street can outperform a newer one elsewhere.

How the numeric features relate to each other

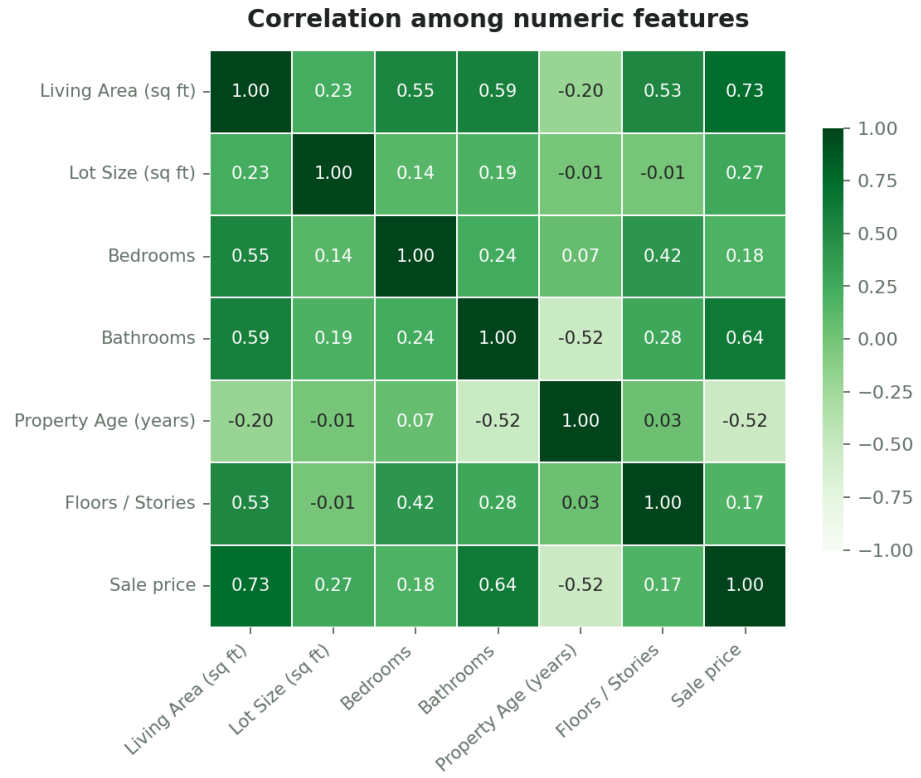


Figure 4. Correlation matrix for the numeric features. Living area has the strongest linear relationship with price. Bedrooms correlates only weakly - a useful reminder that a feature people assume matters does not always carry much signal.

Location sets the ceiling

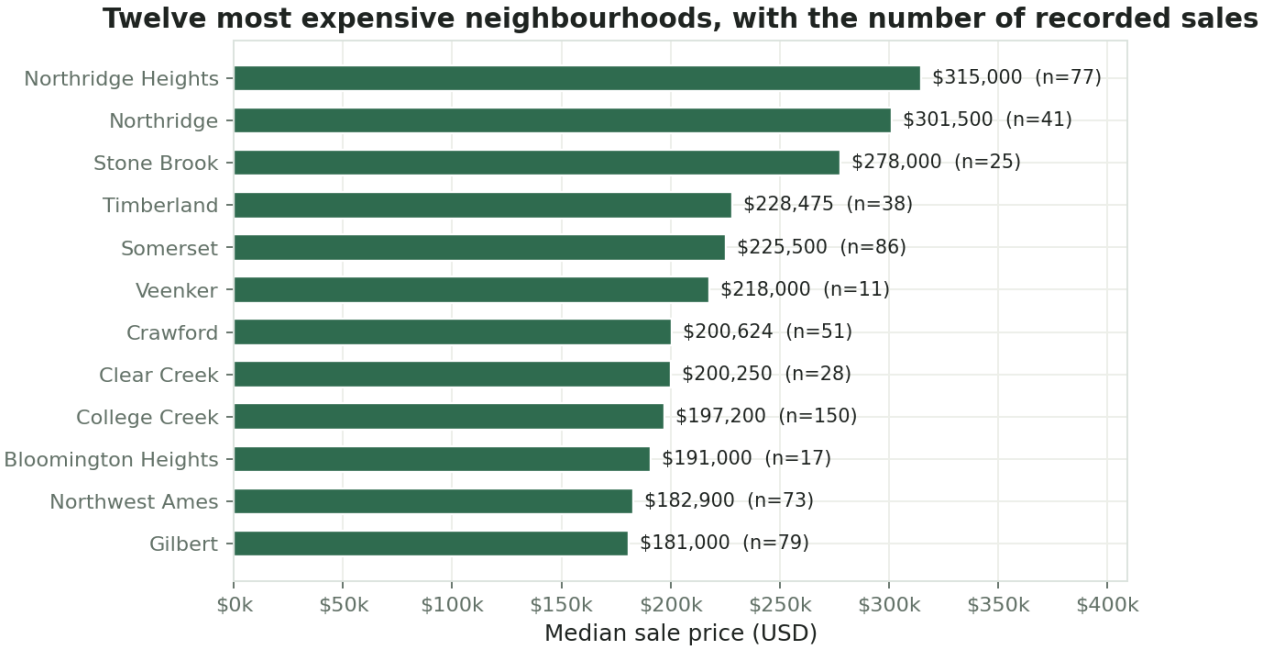


Figure 5. The twelve most expensive neighbourhoods, with the number of recorded sales in each. Some neighbourhoods have fewer than a dozen sales, which is why per-neighbourhood estimates for them are noisy - see Chapter 23.

5. Cleaning: what we removed and exactly why

The raw extract holds 1,460 records. Cleaning leaves 1,458. Only two rows are dropped, and the reason is documented rather than convenient.

Step	Rows affected	Why
Drop the row identifier	0 removed	An Id column carries no predictive signal.
Remove exact duplicates	0 removed	Duplicated rows would weight some properties twice. None were found.
Remove non-positive area or price	0 removed	A zero or negative price or area indicates a broken record. None were found.
Remove documented partial sales	2 removed	Two sales over 4,000 sq ft sold for under $300,000. The dataset author documents both as partial sales - incomplete construction, not arm's-length market transactions - so they are not valid training data.

Deliberately left alone: 6 missing bedroom values

Six records have a bedroom count of zero, which is not a real studio - it is missing data recorded as zero. The code converts these to genuinely missing values and leaves them that way. They are filled later by an imputer that is fitted **only on the training split**. Pre-filling them across the whole dataset would let information from the test set leak into training, and would make the reported accuracy optimistic and dishonest.

6. Feature engineering: building the ten-column schema

Raw columns are not always the right shape to learn from. Four features are constructed rather than taken as-is.

Feature	How it is built	Why it is better than the raw column
bathrooms	FullBath + 0.5 x HalfBath + BsmtFullBath + 0.5 x BsmtHalfBath	Four separate bathroom columns collapse into one number a person would recognise.
property_age	YrSold - YearBuilt, clipped at zero	Age at the moment of sale, not age today. Two sale years differ in what a construction year means; age removes that ambiguity.
floors	HouseStyle mapped to storeys (1, 1.5, 2, 2.5)	Turns a text label into an ordered number, so 1.5 sits between 1 and 2.
parking	GarageCars, with a missing garage type treated as no garage	Capacity is what buyers care about. A missing garage type means no garage, not unknown.
location	Neighbourhood codes mapped to names	NAmes means nothing to a user; 'North Ames' does.
property_type	BldgType mapped to plain English	1Fam becomes 'Single-Family Detached'.

A note on bathrooms

Counting a half bath as 0.5 rather than as a separate category is a modelling choice. It keeps the feature numeric and ordinally meaningful - three full baths beats two full plus a half - which suits both algorithms used here.

7. The train/test split, and why leakage ruins everything

The dataset is split before any fitting happens: **1,166 training rows** and **292 held-out test rows** (80/20), with a fixed random seed of 42 so the split is identical on every run.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

The model sees only the training rows. The test rows are locked away until the very end, and are touched exactly once - to produce the numbers in Chapter 15.

What data leakage is, and why the pipeline avoids it

Leakage happens when information from the test set sneaks into training. It makes measured accuracy look excellent and then collapses in the real world. The classic route is computing a median or fitting a scaler across the whole dataset before splitting. This project avoids it by putting every transform - imputation, scaling, encoding - inside a scikit-learn Pipeline, so they are fitted on training rows only.

The pattern that keeps it safe

```
# CORRECT - the imputer learns the median from training rows only
pipeline.fit(X_train, y_train)      # learns medians AND the model
pipeline.predict(X_test)            # applies them, no refitting

# WRONG - the median now includes test rows
df['bedrooms'].fillna(df['bedrooms'].median())   # before the split
train_test_split(...)
```

Part III - The algorithms

This is the heart of the project. Two candidate algorithms were trained and compared. This part explains how each one works before revealing which won.

8. Algorithm one: Linear Regression

The oldest and simplest regression method, and still the right place to start. It assumes price is a weighted sum of the inputs.

```
price = w1*area + w2*bedrooms + w3*age + ... + b

  wi  = how many dollars one extra unit of that feature is worth
  b   = the baseline price when every feature is zero
```

Training means finding the weights. The method used is **ordinary least squares**: pick the weights that make the sum of squared errors as small as possible.

```
           n
  minimise  SUM (actual_i - predicted_i)^2
           i=1

Squaring the error punishes big misses far more than small ones,
and gives one unique best answer that can be solved directly.
```

What it does well

- Fast - training takes a fraction of a second even on large data.
- Completely transparent: each weight reads as a dollar effect you can quote.
- Hard to overfit badly, because there are so few parameters to fit.

Where it struggles

- **It assumes each unit adds a fixed amount.** If an extra bathroom adds $8,000, the model says it adds $8,000 whether the house is 900 or 4,000 square feet. Reality is not that tidy.
- **It cannot discover interactions on its own.** A garage matters more on a large family home than on a small one. Capturing that means adding a hand-built area x parking term - and you have to guess which interactions matter.
- **It is sensitive to extreme values.** Squaring errors means a handful of expensive homes can drag the whole fit.

In this project the linear model is the **baseline**: the honest reference point that any fancier method has to beat to justify its extra complexity.

9. Algorithm two: Decision Trees

A decision tree predicts by asking a sequence of yes/no questions about the features, then reading the answer off the group it lands in.

```
Is the living area above 1,600 sq ft?
  |-- NO:  Is the property older than 40 years?
  |         |-- NO:  -> predict $152,000
  |         |-- YES: -> predict $118,000
  |-- YES: Is it in Northridge Heights, Northridge or Stone Brook?
            |-- NO:  -> predict $196,000
            |-- YES: -> predict $312,000
```

Training works top-down. At each step the tree searches every feature and every possible split point, and picks the one that best separates high prices from low ones - measured by how much it reduces the variance of the price within each group. It stops when a branch is small enough, deep enough, or cannot be split usefully.

Why a single tree is not enough

A deep tree memorises its training data. It will nail every house it has already seen and generalise poorly to anything new. This is **overfitting**, and a lone unrestrained tree is extremely prone to it. A second weakness: trees are unstable. Change a handful of training rows and you can get a completely different tree.

10. Algorithm three: Random Forest

A random forest is what you get when you fix both of those problems at once. It builds many trees and averages their predictions.

Two ideas do the work

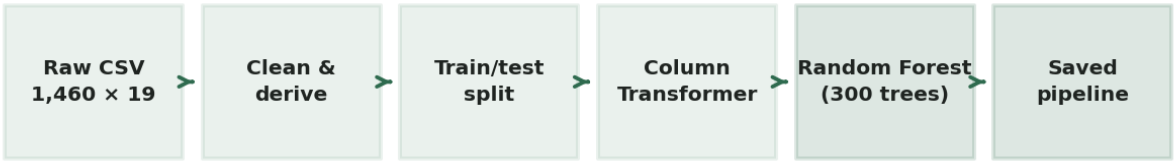
1. **Bagging (bootstrap aggregating).** Each tree is trained on a random sample of the training rows, drawn with replacement - so a given row may appear several times in one tree's sample and not at all in another's. Every tree therefore sees a slightly different world.
2. **Random feature subsets.** At each split, a tree may only consider a random subset of the features rather than all nine. This stops every tree from latching onto the same obvious feature (here, living area) and forces genuinely different trees.

Then it averages. Each tree votes with a price; the forest's answer is the mean. Individual errors - one tree over-predicting, another under - cancel out, while the shared signal survives.

> **Why averaging helps, in one sentence**
>
> Averaging reduces **variance** without systematically increasing **bias**: the forest keeps the trees' ability to fit complex patterns while smoothing away their tendency to memorise noise. Averaging only works because the individual trees are decorrelated - that is precisely what the random row and feature sampling buys.

What this project actually configured

The forest is fitted inside the same Pipeline that handles imputation, scaling and encoding, so the transforms are fitted on training rows only. The pipeline is then saved as a single artifact and loaded once at application start-up - no refitting per request.



Everything inside one Pipeline — the same fitted transforms apply at inference

Figure 6. The full pipeline, from raw CSV to a saved model artifact. Everything lives inside one Pipeline object, so the same fitted transforms are guaranteed to apply at inference time.

11. Which algorithm was chosen, and why

The selected model is the Random Forest Regressor. It was not chosen in advance and it was not chosen on training performance - both models were fitted, then compared on the 292 held-out test rows that neither had seen.

The head-to-head, on held-out data

Metric	Linear Regression	Random Forest	Better	Improvement
MAE	$21,974.48	$19,039.07	Random Forest	13.4% lower
RMSE	$30,378.88	$27,040.66	Random Forest	11.0% lower
R-squared	0.8329	0.8676	Random Forest	higher
MAPE	13.87%	12.18%	Random Forest	lower

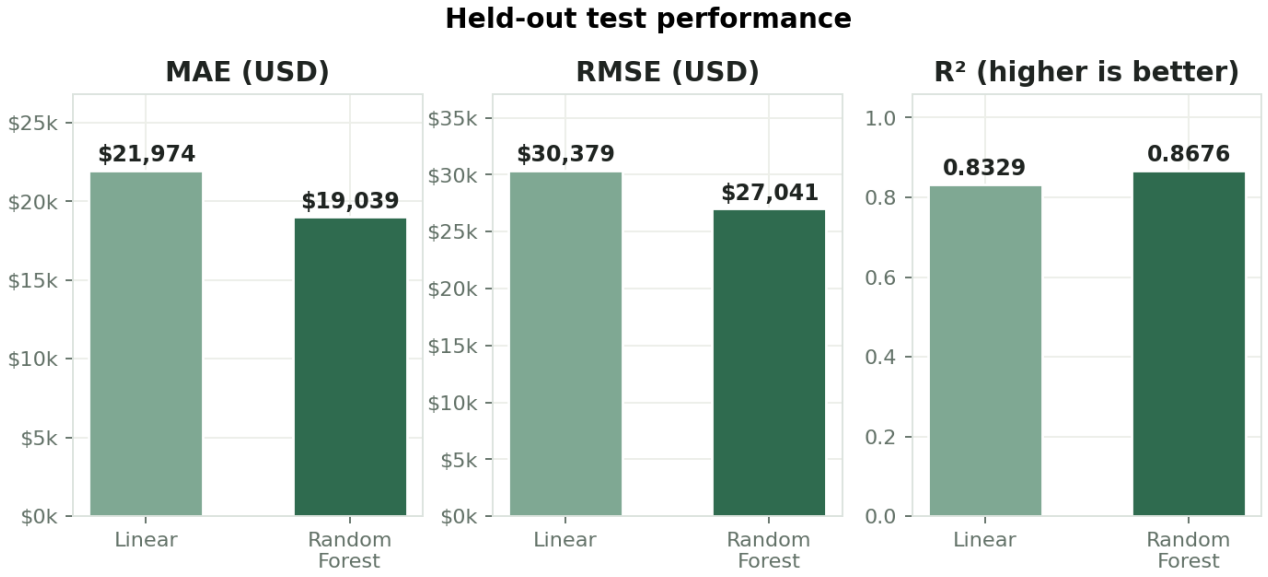


Figure 7. Held-out test performance. The Random Forest wins on all four metrics, not just one - which is what makes the choice straightforward rather than a matter of taste.

The reasoning, in plain terms

1. **It wins on every metric, not a selected subset.** When one model wins on MAE, RMSE, R-squared and MAPE simultaneously, there is no trade-off to argue about.
2. **The problem is Interaction-shaped.** Garage capacity matters more on a large house than a small one; location and area combine rather than simply add up. Forests capture that structure automatically. A linear model would need those interaction terms hand-written, and guessing them wrong is easy.
3. **Categorical features with many levels.** Location has 25 values. One-hot encoding plus a linear model fits one coefficient per neighbourhood and nothing else - it cannot express that a large house in a good neighbourhood is worth more than the sum of those two facts.
4. **Robustness to the price tail.** Sale prices are right-skewed (Figure 1). Trees split on ranks rather than magnitudes, so a handful of $700,000 homes distort the fit less than they distort a least-squares line.
5. **The accuracy cost is worth it.** The forest is slower to train and its individual predictions are harder to explain than a linear equation. At this scale - 1,458 rows, nine features - that cost is negligible next to an 11 percent cut in test error.

Where we would have preferred the linear model

Honesty cuts both ways. Linear regression would be the better engineering choice if:

- Every prediction had to be explained as a dollar breakdown - 'this bathroom added $8,000'.
- The dataset were far smaller, where a forest's extra capacity would overfit.
- The relationships were genuinely straight and additive, in which case the forest buys nothing.
- Training speed or memory were the binding constraint.

None of those hold here. Notably, the linear baseline is a **strong** one - test R-squared of 0.8329 - so beating it is a real result rather than a straw-man comparison.

The generalisation gap, stated openly

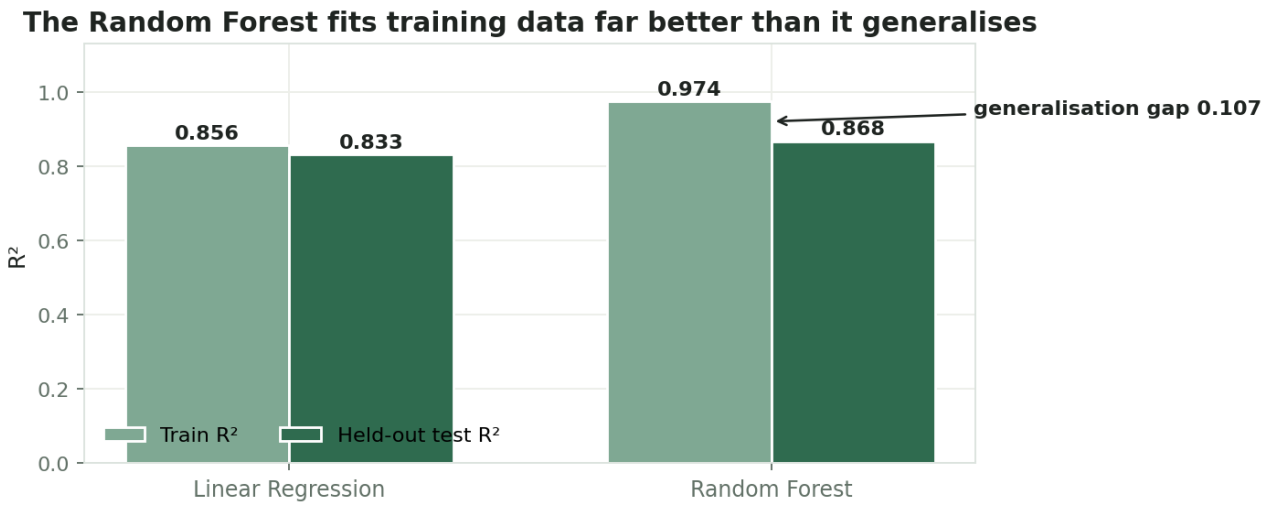


Figure 8. Train versus held-out test R-squared. The forest reaches 0.9743 on training data but 0.8676 on held-out data - a gap of 0.1067. It has memorised its training set to a degree, and the honest number is the test one. Closing that gap is the first item in Chapter 24.

12. The preprocessing pipeline in detail

Both models sit inside a scikit-learn **ColumnTransformer**, which routes numeric and categorical columns down different paths.

Column type	Columns	Steps applied	Reason
Numeric	area_sqft, lot_area_sqft, bedrooms, bathrooms, property_age, floors	SimpleImputer (median), then StandardScaler	Fills the missing bedroom values; scaling stops features measured in thousands of square feet from dominating those measured in single digits.
Categorical	location, parking, property_type	SimpleImputer (most frequent), then OneHotEncoder	Converts text labels into 0/1 indicator columns so the maths can use them.

> **Why scaling matters even for the forest**
>
> Tree splits are based on ordering, so scaling is not strictly required for the forest itself. It is included because the same pipeline structure is shared with the linear baseline, where unscaled features make the coefficients impossible to compare. Keeping one pipeline for both models makes the comparison fair.

The shape of the code

```
preprocessor = ColumnTransformer([
    ('num', Pipeline([('impute', SimpleImputer(strategy='median')),
                      ('scale',  StandardScaler())]),
     NUMERICAL_FEATURES),
    ('cat', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                      ('encode', OneHotEncoder(handle_unknown='ignore'))]),
     CATEGORICAL_FEATURES),
])

model = Pipeline([('prep', preprocessor), ('model', RandomForestRegressor())])
```

`handle_unknown='ignore'` is deliberate. If a category appears at prediction time that was never seen in training, the encoder produces all zeros instead of raising. The validation layer rejects unknown values before they get that far, but this is the safety net behind it.

Part IV - Evaluation

13. How accuracy is measured here

Four metrics, because any single one can be misleading. All are computed on the 292 held-out rows.

Metric	What it is	How to read it	Why it is reported
MAE	Mean absolute error	$19,039 - the average miss in dollars	The most intuitive number. Easy to explain to anyone.
RMSE	Root mean squared error	$27,041 - like MAE, but punishes large misses harder	Reveals occasional big errors that MAE hides. Always larger than MAE; a large gap between them means some predictions are badly off.
R-squared	Share of price variance explained	0.8676 - about 87% of the variation	Scale-free, so it is comparable across datasets. 1.0 is perfect; 0.0 is no better than always guessing the average.
MAPE	Mean absolute percentage error	12.18% - the average miss as a share of price	Puts the error in proportion. A $19,000 miss means something different on a $60,000 home than on a $600,000 one.

The single most important caveat

An MAE of $19,039 is an **average** over 292 held-out sales. It is a fair summary of the system, and it is **not a guarantee about any individual property**. A specific home can be much further off. This is one reason the application deliberately does not display a prediction interval - see Chapter 23.

14. Results

Random Forest - the selected model

Metric	Training	Held-out test
MAE	$8,324.85	$19,039.07
RMSE	$12,949.28	$27,040.66
R-squared	0.9743	0.8676
MAPE	4.90%	12.18%

Linear Regression - the baseline

Metric	Training	Held-out test
MAE	$21,103.68	$21,974.48
RMSE	$30,609.74	$30,378.88
R-squared	0.8561	0.8329

Metric	Training	Held-out test
MAPE	12.68%	13.87%

Notice the pattern in the two tables. The linear model's training and test numbers are close together - it cannot overfit much, because it has so little freedom. The forest's training numbers are far better, and its test numbers are better too, but by a smaller margin. Both facts are reported rather than hidden.

Seeing the predictions

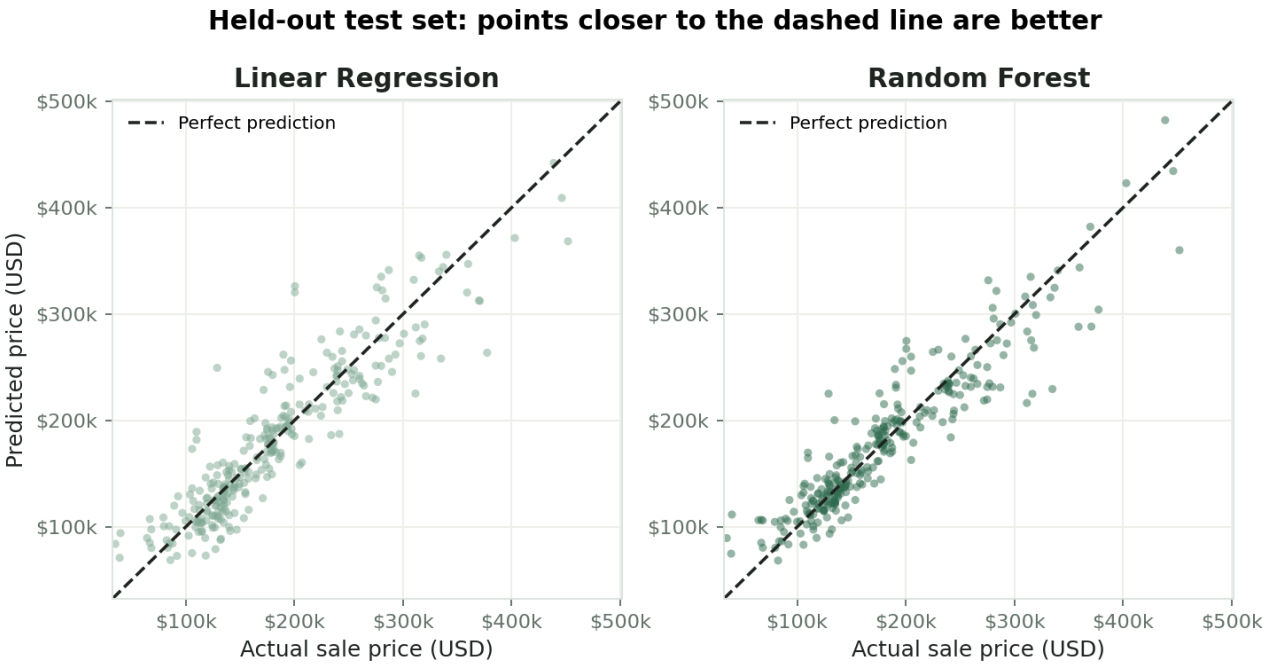


Figure 9. Predicted against actual price on the held-out test set for both models. Points scatter around the dashed line of perfect prediction. The Random Forest cloud is tighter, especially at the expensive end where the linear model systematically under-predicts.

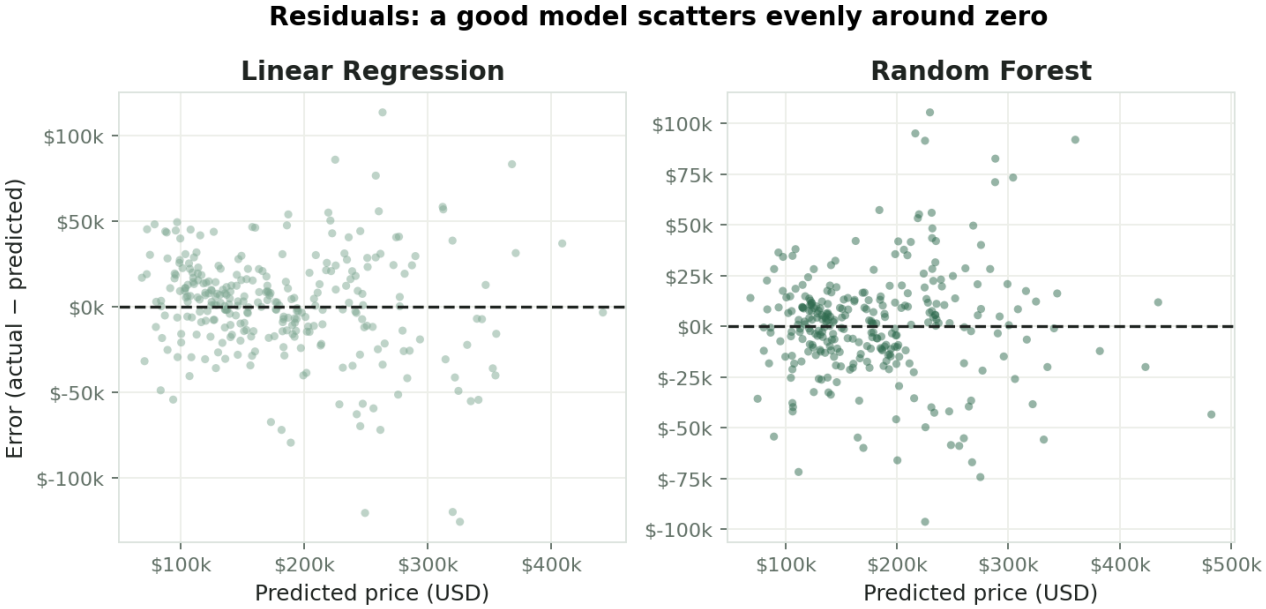


Figure 10. Residuals - the error on each test property. A well-behaved model scatters evenly around zero with no curved shape. Both models fan out at higher prices, which is expected: expensive homes vary more, so there is more to get wrong.

15. What the model learned

Two different importance measures are reported, and they disagree. That disagreement is worth understanding rather than smoothing over.

Feature	Impurity importance	Permutation importance
Parking / Garage	35.11%	9.36%
Living Area (sq ft)	30.87%	48.70%
Property Age (years)	13.45%	26.65%
Bathrooms	6.99%	3.66%
Lot Size (sq ft)	5.22%	3.30%
Neighborhood / Location	3.73%	2.08%
Bedrooms	2.54%	3.25%
Floors / Stories	1.26%	2.22%
Property Type	0.83%	0.79%

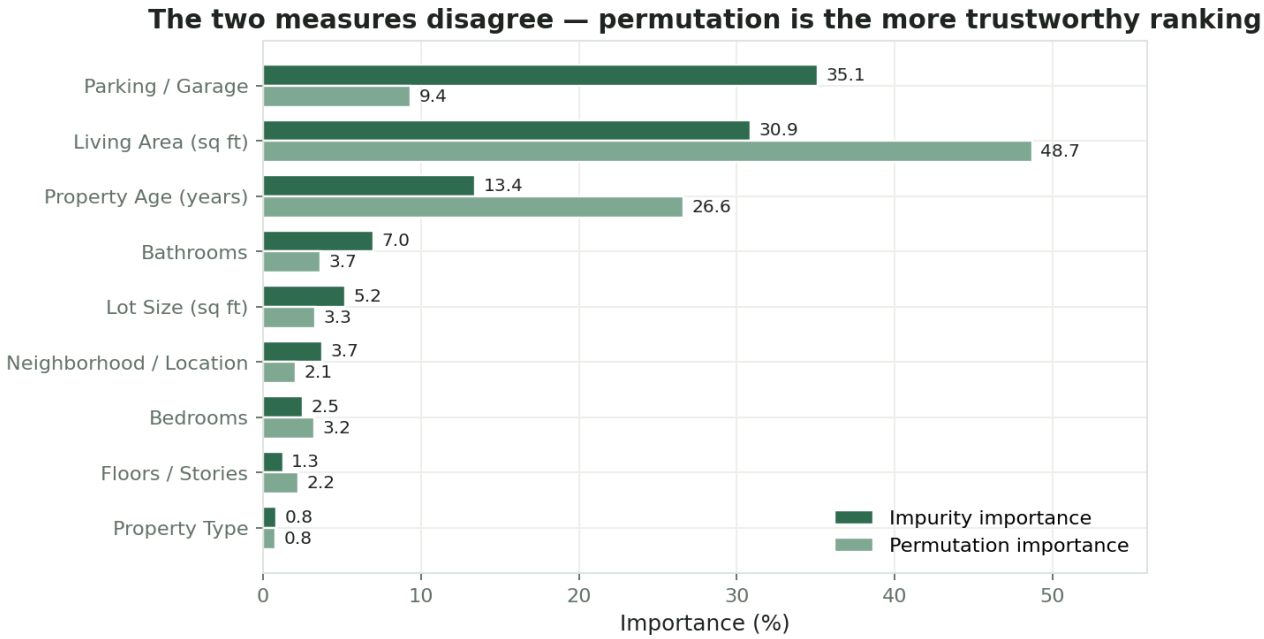


Figure 11. Impurity importance against permutation importance. The two rankings disagree sharply on parking - and permutation is the one to trust.

Why they disagree, and which to believe

Impurity importance counts how often a feature is used to split, weighted by how much those splits reduce variance. It is computed on training data and it is biased toward features with many distinct values - a feature with four categories offers more places to split than one with two, so it gets more chances to look useful.

Permutation importance shuffles one feature's values across the held-out set and measures how much worse the predictions get. It is measured on data the model has not memorised, and it asks the question we actually care about: how much does this feature contribute to real predictive accuracy?

The verdict

Impurity importance ranks parking first at 35.11 percent; permutation ranks it at 9.36 percent. Permutation is measured on held-out data and is not biased by category count, so it is the more trustworthy guide. It puts **living area first at 48.70 percent** and **property age second at 26.65 percent** - which also matches how housing markets actually behave.

These numbers are not causal

Feature importance tells you which inputs the model relies on. It does not say that adding a garage *causes* a price increase, and it must never be read that way. Adding a garage to a house would not necessarily raise its value by anything - the model has learned an association in past sales data, not a mechanism.

Part V - The system

16. How a prediction request flows

The application is a small Flask service. Understanding the path a request takes explains most of the design decisions.

```
Browser                nine inputs, checked instantly in the browser
   |
   v
POST /api/predict      the same rules are enforced again on the server
   |
   v
validate_and_convert   ranges, types and known category values
   |
   v
predict_price          the pipeline, loaded once and cached in memory
   |
   +--> prediction_store   every estimate is recorded to SQLite
   |
   v
JSON response          price, price per sq ft, explanation, limitations
```

Concern	How it is handled
Model loading	Loaded once at start-up and cached. A reload per request would be wasteful and slow.
Validation	Identical rules run in the browser and on the server. Client-side checks are a convenience, never a guarantee - they can be bypassed.
Unknown categories	Rejected with a specific message. The system never guesses at a neighbourhood or property type it was not trained on.
Logging	Every served estimate is written to SQLite. If logging fails, the prediction still succeeds - a logging problem must never break a valid answer.
Serving	Flask's built-in development server. For anything beyond local use, deploy behind Gunicorn or Waitress.

17. The web application

Page	Route	What it does
Landing	/	Headline, two calls to action, and the key dataset facts.
Estimate form	/predict	Nine inputs, pre-filled with the training medians so a first visit can produce a sensible result immediately.
Result	/result	Estimated price, price per square foot, a sensitivity table showing how the estimate moves when one input changes, and the limitations stated alongside the number.
Insights	/insights	Six charts generated by Matplotlib and Seaborn from the real dataset and fitted model.
Dashboard	/dashboard	Live view of the predictions this installation has served (Chapter 18).

The defaults in the form are the training-set medians: 1,464 sq ft of living area, 9,550 sq ft of lot, 3 bedrooms, 2 bathrooms, 36 years old, 1 floor. They are computed from the training split, not the full dataset - the same no-leakage discipline as everywhere else.

> **The sensitivity table**
>
> The result page shows how the estimate changes when one input is nudged while the rest stay fixed. This is an exploratory aid, not a causal claim: it shows what the model says about a hypothetical variant of the property, not what a renovation would achieve.

18. The prediction dashboard

The dashboard answers the question 'what has this installation actually been asked to price?'. It is built on a SQLite log at `data/predictions.db`, with one row per served estimate.

Two different things, kept visibly separate

	Usage	Measured accuracy
Source	The SQLite log of served predictions	models/metrics.json, from the held-out test set
Changes over time	Yes - grows with every prediction	No - fixed until the model is retrained
Examples	Predictions logged, median estimate, distribution, busiest neighbourhoods	MAE $19,039, R-squared 0.8676

This separation is the point. It would be easy - and dishonest - to let impressive accuracy figures appear to be a property of recent activity, or to let a busy day make the model look better. They are unrelated quantities and the page keeps them apart.

What it shows once there is data

- A KPI strip: predictions logged, median estimate and range, median price per square foot, test R-squared, and typical error in dollars.
- Prediction volume over time, with the median estimate overlaid. Buckets are hourly while activity spans a single day, and switch to daily once it spans more.
- A twelve-bin histogram of logged estimates.
- Median estimate by neighbourhood and by property type, ranked.
- Living area against estimated price, one point per logged estimate, coloured by property age.
- Feature importance, labelled as association rather than causation.
- Model performance, measured once on the held-out test set.
- The twelve most recent predictions the model served.

Nothing on the dashboard is invented

With an empty log the page shows an explicit empty state - not placeholder charts with made-up numbers. The 'Log sample activity' button is not a demo shortcut: it takes real properties from the cleaned dataset, runs each through the saved pipeline exactly as a web request would, and records the result tagged as a sample run so it is always distinguishable from genuine form submissions. All seven charts are hand-rolled inline SVG - no charting library, no CDN, and they work offline.

19. Validation and error handling

Validation runs twice: inline in the browser for instant feedback, then again on the server. The duplication is intentional - a client-side check is a usability feature, not a security boundary.

Failure mode	Behaviour
Missing field	Field-level message naming the field, plus a summary banner.
Out-of-range value	Rejected with the accepted range stated explicitly.
Wrong type	Rejected with a conversion message rather than a crash.
Unknown category	Rejected, naming the value and the valid options.
Model artifact missing	A clear error page explaining how to train the model.
Logging failure	Logged as a warning; the prediction is still returned.

20. Testing

The suite has 70 tests, all passing.

Group	Tests	Covers
Preprocessing	part of 49	The 19-to-10 transformation, removed rows, derived features, missing-value handling.
Inference and routes	part of 49	Valid, invalid, missing, out-of-range and unseen-category inputs; every page route; the JSON API contract; determinism.
Dashboard	21	The SQLite store round-trip, aggregation correctness against the stored rows, the empty-state guarantee, and all three dashboard endpoints.

A test-safety detail worth copying

An autouse session fixture redirects the prediction log to a temporary database and asserts that nothing points it back at the real one. Without it, a test exercising 'clear the log' would destroy genuinely recorded activity. This was a real defect during development and the fixture exists because of it.

21. Running the project yourself

From the project root, with Python 3.9 or newer:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python -m src.data_preprocessing   # fetch OpenML data -> cleaned CSV
python -m src.train_model          # train both models, save the winner
python app.py                      # serve on http://127.0.0.1:5000

python -m pytest tests/ -q         # 70 tests
```

Then open	You will see
http://127.0.0.1:5000/	The landing page
http://127.0.0.1:5000/predict	The estimate form
http://127.0.0.1:5000/insights	The dataset charts
http://127.0.0.1:5000/dashboard	The prediction-activity dashboard

The dashboard starts empty by design. Use the 'Log sample activity' button on that page, or make a few predictions through the form, and it populates with real model runs.

Part VI - Limits and next steps

22. Project structure

```
house-price-prediction/
  app.py                     Flask app: routes, API, error handling
  config.py                  Paths, feature schema, code maps, validation rules
  src/
    data_preprocessing.py    Load, inspect, clean, build the ColumnTransformer
    train_model.py           Train, compare, select, persist
    evaluate_model.py        Metrics, importances, statistics, charts
    predict.py               Validation, cached inference, explanations
    prediction_store.py      SQLite log of every served estimate
    dashboard_analytics.py   Aggregates the log for the dashboard
    sample_activity.py       Scores real dataset rows to seed the log
  data/raw/                  The 19-column extract from OpenML
  data/processed/            The 10-column modelling schema
  models/                    The fitted pipelines and metrics.json
  templates/                 Jinja2 pages, including dashboard.html
  static/css, static/js      Hand-written stylesheet and behaviour
  tests/                     70 pytest tests
  docs/                      This guide, the PDF, the presentation, charts
```

23. What this system cannot do

These limits are not footnotes. They define what the output may legitimately be used for, and the application restates them on the result page.

Limitation	What it means in practice
Geography and period	All training records are from Ames, Iowa, United States between 2006 to 2010. The model knows nothing about other markets, and nothing about price levels after 2010.
Not a valuation	The output is a statistical estimate for education and comparison. It is not an appraisal, not financial advice, and not a substitute for a professional valuation.
No prediction interval	The model emits a single number. A range is deliberately not shown, because a standard Random Forest does not produce a statistically justified one. Quantile regression forests or conformal prediction would be required.
Averages hide individual error	MAE of $19,039 is an average across 292 held-out sales. Any single property can be much further off.
Small categories	Some neighbourhoods have only a handful of recorded sales, so their estimates are noisy (see Figure 5).
Omitted factors	Condition, renovation quality, interior finish, school catchment and micro-location are absent from the nine features. They matter, and the model cannot see them.
Overfitting headroom	Train R-squared 0.9743 against test R-squared 0.8676 leaves clear room for regularisation.
Development server	app.py uses Flask's built-in server. Use Gunicorn or Waitress for anything beyond local use.

24. What would be built next

Change	What it would improve	Why it helps
Hyperparameter tuning with cross-validation	Accuracy	The forest was trained with sensible defaults, not searched settings. Tree depth and minimum leaf size alone are likely worth something.
Regularisation / depth limits	Generalisation	Directly targets the train-test gap in Figure 8.
Log-transform the target	Accuracy on the expensive tail	Sale prices are right-skewed; modelling log-price often evens the error out.
Target or frequency encoding for location	Accuracy	25 neighbourhoods produce many sparse one-hot columns. Encoding them by their median price is often stronger.
Gradient boosting (XGBoost, LightGBM)	Accuracy	Boosting fits errors sequentially rather than averaging independent trees, and usually beats a plain forest on tabular data.
Quantile regression forests	Honesty	Would produce a calibrated prediction interval - the single biggest missing capability.
Conformal prediction	Honesty	Gives intervals with a guaranteed coverage rate, without assuming a distribution.
Retraining on current data	Relevance	Would remove the 2006-2010 constraint, at the cost of sourcing a licensed modern dataset.

25. Glossary

Term	Meaning
Bagging	Training each model on a random resample of the data, then averaging. Short for bootstrap aggregating.
Bias	Error from assumptions that are too simple - missing the true pattern.
Classification	Predicting a category.
Data leakage	Information from the test set reaching the model during training, inflating measured accuracy.
Feature	An input column the model learns from.
Feature engineering	Creating better inputs from the raw columns.
Held-out / test set	Data locked away during training and used once to measure honest accuracy.
Hyperparameter	A setting chosen before training, not learned from data.
Imputation	Filling in missing values.
MAE	Mean absolute error - the average miss, in the target's units.
MAPE	Mean absolute percentage error - the average miss as a share of the true value.
Model	The learned rule that maps inputs to a prediction.
One-hot encoding	Turning one categorical column into several 0/1 columns.

Term	Meaning
Overfitting	Memorising training data instead of learning the pattern.
Permutation importance	How much accuracy falls when one feature's values are shuffled.
Pipeline	A chain of transforms plus a model, fitted together.
Regression	Predicting a number.
RMSE	Root mean squared error - like MAE, but punishing large misses more.
R-squared	Share of the target's variance the model explains. 1.0 is perfect.
Random Forest	Many decorrelated decision trees, averaged.
Residual	The error on one prediction: actual minus predicted.
Scaling	Rescaling numeric features so they are comparable in magnitude.
Supervised learning	Learning from examples that include the correct answer.
Target	The value being predicted - here, sale price.
Training set	The examples the model learns from.
Variance	Error from being overly sensitive to the particular training sample.

26. Appendix: JSON API reference

Endpoint	Method	Purpose
/api/predict	POST	Validate nine inputs and return an estimate with an explanation.
/api/metrics	GET	The full metrics bundle as JSON.
/api/dashboard/summary	GET	The complete dashboard payload.
/api/dashboard/sample	POST	Score and log {"count": N} real properties from the dataset (1 to 200).
/api/dashboard/clear	POST	Delete every logged prediction.
/health	GET	Liveness and model-status check.

27. Appendix: the ten-column data schema

This is the required format for retraining. Feed a CSV with these columns, in this order, to `python -m src.train_model`.

#	Column	Type	Description	Example
1	location	text	One of 25 neighbourhoods	College Creek
2	area_sqft	number	Above-grade living area, sq ft	1710.0
3	lot_area_sqft	number	Lot size, sq ft	8450.0
4	bedrooms	number	Bedrooms above grade	3.0

#	Column	Type	Description	Example
5	bathrooms	number	Total bathrooms, half bath = 0.5	3.5
6	property_age	number	Age at the point of sale, years	5.0
7	floors	number	Storeys: 1, 1.5, 2 or 2.5	2.0
8	parking	text	No Garage / 1-Car / 2-Car / 3+ Car	2-Car Garage
9	property_type	text	One of five building types	Single-Family Detached
10	price	number	Target. Sale price in USD	208500.0

If you retrain on different data

Every figure in this guide describes this dataset and this fitted model. Retraining will change all of them. Regenerate the charts and this document afterwards so the write-up cannot drift from the artifacts.

Summary. A reproducible pipeline, a model chosen on held-out data rather than asserted in advance, and an explicit account of what it cannot do. The Random Forest was selected over a strong linear baseline because it won on all four held-out metrics - 11 percent lower RMSE - and because the problem is interaction-shaped in a way a linear model cannot express without hand-written terms. On homes it has never seen it explains about 87 percent of price variation and is typically off by about $19,039.